==========

AWS REAL-TIME SCENARIO-BASED TASKS

==========

SCENARIO 1: E-COMMERCE WEBSITE HIGH AVAILABILITY

Business Problem:

An e-commerce company experiences heavy traffic during festival sales.

The website becomes unavailable when traffic increases.

Objective:

Design a highly available and scalable architecture.

Services:

- EC2
- Application Load Balancer (ALB)
- Auto Scaling Group (ASG)
- CloudWatch
- SNS

Tasks:

1. Launch two EC2 instances in different Availability Zones.
2. Install and configure a web server (Apache/Nginx).
3. Create an Application Load Balancer.
4. Register EC2 instances with the ALB.
5. Create a Launch Template.
6. Create an Auto Scaling Group.
7. Configure scaling policies:
   - Scale Out: CPU > 70%
   - Scale In: CPU < 30%
8. Create CloudWatch Alarms.

9. Configure SNS email notifications.

10. Simulate high traffic and verify auto scaling.

Expected Outcome:

- Traffic is distributed across servers.

- New instances launch automatically.

- Email alerts are received.

- No downtime occurs if one server fails.

==========

SCENARIO 2: SECURE 3-TIER FINTECH APPLICATION

==========

Business Problem:

A fintech startup requires a secure architecture where the database

must not be accessible from the internet.

Objective:

Build a secure 3-tier architecture.

Services:

- VPC

- EC2

- RDS

- NAT Gateway

- Security Groups

- NACL

Tasks:

1. Create a custom VPC.

2. Create:

   - 2 Public Subnets

   - 2 Private Subnets

3. Deploy Bastion Host in Public Subnet.

4. Deploy Application Server in Private Subnet.

5. Deploy MySQL RDS in Private Subnet.

6. Configure NAT Gateway.

7. Configure Route Tables.

8. Configure Security Groups:

   - SSH only through Bastion Host

   - RDS accessible only from App Server

9. Configure NACL rules.

10. Verify secure connectivity.

Expected Outcome:

- Database remains private.

- Internet access only through NAT Gateway.

- Secure administrative access.

==========

SCENARIO 3: MEDIA COMPANY STORAGE OPTIMIZATION

==========

Business Problem:

A media company stores large volumes of videos and wants to reduce storage costs.

Objective:

Implement cost optimization using S3 Lifecycle Policies.

Services:

- S3

- IAM

Tasks:

1. Create an S3 Bucket.

2. Enable Versioning.

3. Enable Server-Side Encryption.

4. Create Lifecycle Policy:

   - 30 Days → Standard-IA

   - 90 Days → Glacier

   - 365 Days → Deep Archive

5. Upload sample files.

6. Test file recovery using versioning.

7. Create IAM policy for upload access.

Expected Outcome:

- Storage costs reduced.

- Files protected through versioning.

- Data encrypted.

==========

SCENARIO 4: IAM ACCESS MANAGEMENT

==========

Business Problem:

Employees are sharing the AWS Root Account.

Objective:

Implement proper Identity and Access Management.

Services:

- IAM

Tasks:

1. Create IAM Groups:
   - Developers
   - Network Team
   - Security Team
2. Create IAM Users.
3. Assign users to groups.
4. Create custom policies:
   - Developers → EC2 Access
   - Network Team → VPC Access
   - Security Team → IAM Full Access
5. Enable MFA for all users.
6. Test permissions.

Expected Outcome:

- Principle of Least Privilege implemented.
- MFA enabled.
- Secure access control.

==========

SCENARIO 5: EBS BACKUP AND DISASTER RECOVERY

==========

Business Problem:

Critical customer data is stored on EBS volumes.

Objective:

Implement backup and recovery procedures.

Services:

- EBS
- Snapshot
- EC2

Tasks:

1. Create EBS Volume.
2. Attach volume to EC2.
3. Store sample data.
4. Create EBS Snapshot.
5. Delete volume.
6. Restore volume from Snapshot.
7. Reattach volume.
8. Verify data integrity.

Expected Outcome:

- Successful recovery from backup.
- Minimal downtime.

==========

SCENARIO 6: SHARED FILE STORAGE FOR APPLICATION SERVERS

==========

Business Problem:

Multiple EC2 instances need access to the same uploaded files.

Objective:

Implement shared storage.

Services:

- EFS

- EC2

Tasks:

1. Launch two EC2 instances.

2. Create EFS.

3. Mount EFS on both servers.

4. Create files from Server-1.

5. Verify visibility from Server-2.

6. Test concurrent access.

Expected Outcome:

- Shared storage across servers.

- Real-time synchronization.

==========

SCENARIO 7: DATABASE MIGRATION TO AWS

==========

Business Problem:

Company wants to move MySQL database from on-premises to AWS.

Objective:

Deploy and manage RDS.

Services:

- RDS

- EC2

Tasks:

1. Create MySQL RDS instance.

2. Configure Security Groups.

3. Launch EC2 instance.

4. Connect EC2 to RDS.

5. Create database and tables.

6. Import sample data.

7. Configure backups.

8. Enable Multi-AZ deployment.

Expected Outcome:

- Highly available managed database.

- Automated backups.

==========

SCENARIO 8: EVENT-DRIVEN ORDER PROCESSING

==========

Business Problem:

An e-commerce platform needs asynchronous communication between services.

Objective:

Implement SNS and SQS architecture.

Services:

- SNS
- SQS

Tasks:

1. Create SNS Topic.
2. Create:
   - Inventory Queue
   - Billing Queue
   - Notification Queue
3. Subscribe queues to SNS Topic.
4. Publish order messages.
5. Verify message delivery.
6. Simulate service failures.

Expected Outcome:

- Reliable event-driven communication.
- Message durability.

==========

SCENARIO 9: CENTRALIZED MONITORING SYSTEM

==========

Business Problem:

Operations team needs proactive monitoring.

Objective:

Implement monitoring and alerting.

Services:

- CloudWatch
- SNS

Tasks:

1. Create CloudWatch Dashboard.
2. Monitor:
   - CPU
   - Memory
   - Disk Usage
   - Network Usage
3. Create CloudWatch Alarms.
4. Configure SNS notifications.
5. Test alert generation.

Expected Outcome:

- Centralized monitoring.

- Immediate alert notifications.

==========

SCENARIO 10: ROUTE 53 FAILOVER IMPLEMENTATION

==========

Business Problem:

Company requires disaster recovery across regions.

Objective:

Implement DNS failover routing.

Services:

- Route 53
- EC2

Tasks:

1. Deploy Web Server in Mumbai Region.
2. Deploy Backup Server in Singapore Region.
3. Create Hosted Zone.
4. Create Health Check.
5. Configure Failover Routing Policy.
6. Simulate primary server failure.
7. Verify automatic failover.

Expected Outcome:

- Automatic traffic redirection.
- Improved availability.

==========

FINAL CAPSTONE PROJECT

==========

Project Name:

Online Shopping Application Infrastructure

Business Requirement:

Build a production-ready AWS environment capable of handling
high traffic while maintaining security, availability,
monitoring, and disaster recovery.

Services Required:

- IAM
- VPC
- EC2
- EBS
- EFS
- ALB
- Auto Scaling
- RDS
- SNS
- SQS
- CloudWatch
- Route 53

Implementation Tasks:

NETWORKING

---

1. Create Custom VPC.
2. Create:
   - 2 Public Subnets
   - 2 Private Subnets
3. Configure Internet Gateway.
4. Configure NAT Gateway.
5. Configure Route Tables.

COMPUTE

---

1. Create Launch Template.
2. Deploy Auto Scaling Group.
3. Deploy EC2 instances in Private Subnets.

LOAD BALANCING

---

1. Create Application Load Balancer.
2. Configure Target Group.
3. Attach Auto Scaling instances.

STORAGE

---

1. Create EBS volumes.
2. Create EFS shared storage.

DATABASE

---

1. Deploy MySQL RDS in Private Subnet.
2. Enable Automated Backups.
3. Enable Multi-AZ.

SECURITY

---

1. Create IAM Users and Roles.
2. Enable MFA.
3. Configure Security Groups.
4. Configure NACLs.

MONITORING

---

1. Create CloudWatch Dashboard.
2. Create CloudWatch Alarms.
3. Configure SNS Notifications.

APPLICATION SERVICES

---

1. Create SNS Topic.
2. Create SQS Queues.
3. Implement order-processing workflow.

DNS

---

1. Create Hosted Zone.
2. Configure Route 53 records.
3. Point domain to ALB.

DISASTER RECOVERY

---

1. Create EBS Snapshots.
2. Test Snapshot Restore.
3. Configure Route 53 Failover.

FINAL TESTING

---

1. Simulate EC2 failure.
2. Verify Load Balancer operation.
3. Generate CPU load.
4. Verify Auto Scaling.
5. Trigger CloudWatch Alarm.
6. Verify SNS Email.
7. Test database connectivity.
8. Verify EFS file sharing.

Expected Result:

A fully functional production-grade AWS architecture similar to

what is deployed in real-world enterprises.

==========